



Tecnológico de Costa Rica

Campus Central Cartago

Proyecto 2 - Explorador

IC5701 - Compiladores e intérpretes

Profesor: Aurelio Sanabria

II Semestre - 2026

Estudiantes:

Isaac Jiménez Blanco - 2024202804

Andrés Gabriel Padilla Robles - 2025137295

Alina Rodriguez Wong - 2024063444

Neithan Vargas Vargas - 2025149384

Adolfo Barquero Monestel - 2022211388

8 de octubre de 2026

Lecciones aprendidas

Nombre: Adolfo Barquero

Con el desarrollo del explorador del proyecto comprendimos cuál es la importancia de separar cada una de los caracteres de las instrucciones en tokens para poder identificar y clasificar correctamente los elementos del lenguaje como palabras reservadas, identificadores, operadores y símbolos. Además cada uno de los token que se generan deben de contener la mayor información posible, para que en las próximas etapas sean más fáciles de poder desarrollar. Por otro lado, el buen manejo de los errores en esta etapa facilita la identificación y solución de problemas en el código fuente, además de brindar mayor información para las próximas etapas del compilador en caso de que en estas se presente el error. Finalmente aprendimos que el explorador es una parte importante del compilador ya que envía la lista de token que el analizador va a requerir para la generación del árbol de sintaxis abstracta

Nombre: Isaac Jiménez Blanco

Durante el desarrollo del explorador aprendí mejor cómo se realiza el análisis léxico y la importancia de separar cada tarea en funciones pequeñas y claras. También comprendí cómo el programa va recorriendo el código fuente, identifica los distintos elementos del lenguaje y los convierte en tokens con información útil como su tipo, línea y columna. Otro aspecto importante fue entender el manejo de errores y cómo una estrategia de recuperación permite continuar con el análisis sin detener todo el programa por un solo problema. Además, este avance me ayudó a ver con más claridad cómo se conectan las distintas etapas del compilador y por qué el explorador sirve como base para que luego el analizador sintáctico pueda trabajar correctamente.

Nombre: Neithan Vargas Vargas

Conforme avance el proyecto comprendí que, que carajo es una gramática LL1. Esto de ir contando letras de código no es lo mio, y tener buenas prácticas es de suma importancia. Porque si no, uno se pierde en el control de la posición dentro del código. Los errores por 1 en programación son terribles. Siempre hay que tener bien, que solo en un lugar se modifique, solo ahí habrá errores. También tratar los casos “Edge”, o casos ocultos lo antes posible, hay que plantearlos todos, porque uno nunca sabe cuando salga un " ", un “\n” o demás que puedan surgir como dolor posteriormente. Como tal, lo que aprendí durante el proyecto es que separar el explorador en funciones pequeñas facilita demasiado la integración del trabajo, porque cada parte puede encargarse de un tipo de token, o problemita sin afectar directamente a los demás, lo cual lo hace demasiado fácil de entender e implementar :).

Nombre: Alina Rodriguez Wong

Durante el desarrollo de las funciones `escanear_comentario` y `escanear_cadena` aprendí que a pesar de que la función parezca algo muy simple de programar, siempre hay que ponerle mente antes de hacerla para pensar en los casos límites. En este contexto fue que pasaba si la cadena o el comentario nunca cerraban o si estaban vacíos. Siempre se debe pensar en que debe haber cuando se entra a la función y cómo se sale. Con respecto al proyecto en general, aprendí que cada decisión tomada al diseñar la gramática nos afecta en la dificultad para programar estas otras partes del proyecto. Por ejemplo, usar símbolos poco comunes en el lenguaje que ni siquiera están en el teclado de mi compu me obligaron a prestar más atención y nos van a ralentizar a la hora de programar con el lenguaje.

Nombre: Andrés Gabriel Padilla Robles

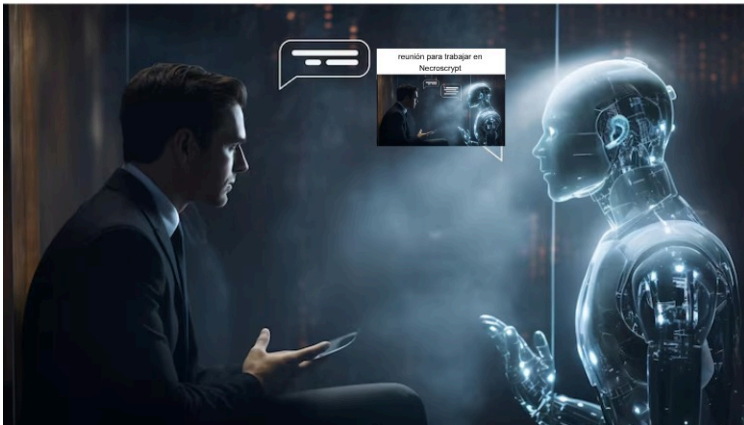
La lección que me llevo es que un compilador no solo tiene que detectar errores, sino que también tiene que decidir cuánto código sacrificar para poder seguir. Si se descarta poco, un solo error puede producir una cascada de errores falsos. Si se descarta mucho, se pierden errores reales que el programador necesitaba ver. Esa decisión depende de cuáles caracteres se definen como seguros.

Memes:

reunión para trabajar en
Necroscript



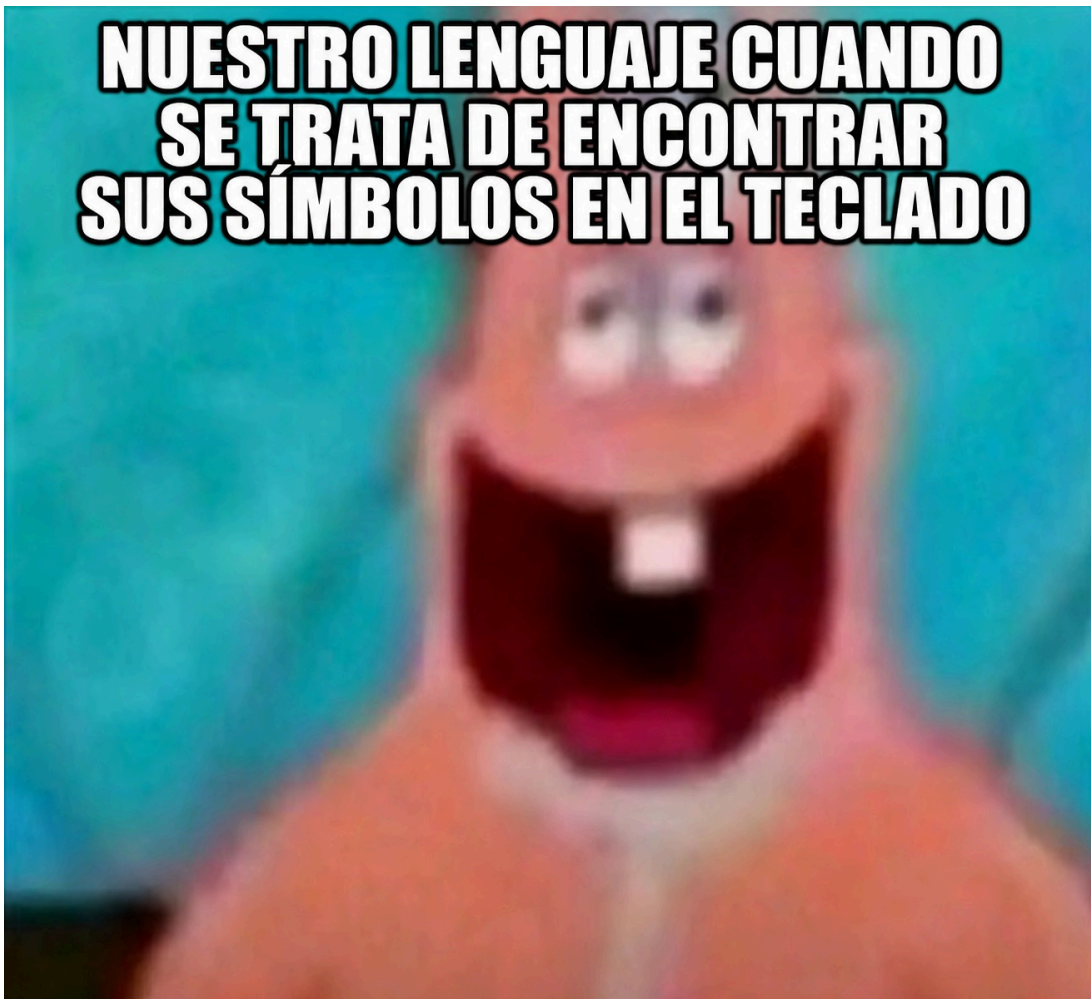
reunión para hacer memes de
Necroscript



Como nos sentimos escribiendo el mejor lenguaje de programación



**NUESTRO LENGUAJE CUANDO
SE TRATA DE ENCONTRAR
SUS SÍMBOLOS EN EL TECLADO**





ritual revisar_destino \ calavera n \ ∅ ∅ cantico



mana \ n restos 2 == 0 \



sacrificar "Alma par" ⊕



sin



sacrificar "Alma impar" ⊕

